

SIMATS ENGINEERING

ASSESSMENT TOOL

COURSE NAME: Theory of Computation **COURSE CODE:** CSA13

COURSE OUTCOME COVERED

CO1: Analyze fundamental mathematical concepts of automata theory for formal languages and decision problems (BL:3)

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:** 25

Duration: 1 Hour

Simulation

Code of Conduct:

I **Shaik Mohammed Waseem Rayan - 192373004** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Shaik Mohammed Waseem Rayan

SIMATS ENGINEERING ASSESSMENT TOOL

- 1.** Design a Non-Deterministic Finite Automaton (NFA) for an Automatic Door System based on sensor inputs. An automatic door system can be modeled using an NFA to represent different states and transitions. The system consists of four states: Closed, Opening, Open, and Closing. The input symbols are P (Person detected) and N (No person detected). Initially, the door is in the Closed state. When a person is detected (P), the system may non-deterministically transition from Closed to Opening. Once opened, the system moves to the Open state. If no person is detected (N), the door transitions to Closing and eventually returns to the Closed state. Non-determinism occurs when the system may remain in the Open state even if a person is detected or may start closing if no detection occurs.
- 2.** Design a finite automaton to model a traffic light control system at a road intersection. The automaton should represent different states corresponding to the traffic lights, such as Red, Green, and Yellow. Define the transitions between these states based on a timer or control signals, ensuring that the sequence follows the standard order (Red → Green → Yellow → Red). Include considerations for timing constraints, such as how long each light remains active, and describe how the automaton handles continuous cycling of signals. Additionally, explain how this model can be extended to handle pedestrian crossing signals or emergency vehicle overrides, ensuring safe and efficient traffic flow.
- 3.** Design DFA using simulator to accept the string that end with ab over set {a,b}. $W = aaabab$
- 4.** Design a Deterministic Finite Automaton (DFA) using a simulator to accept only the input strings “c”, “bc”, and “bcaaa” over the alphabet {a, b, c}. The automaton should begin from an initial state and transition through intermediate states based on the sequence of input symbols, ensuring that each valid string follows a unique path. For example, the string “c” should be accepted directly from the start state, while “bc” should pass through states representing “b” and then “c”. Similarly, “bcaaa” should be processed through a sequence of states that track each additional “a” after “bc”. Any deviation from these exact patterns should lead to a dead state, ensuring that no other strings are accepted. Clearly define the states, transition functions, start state, and accepting states, and implement the DFA in a simulator such as Autosim to verify its correctness.
- 5.** Design a Deterministic Finite Automaton (DFA) using a simulator to accept all strings over the alphabet {a, b} that begin with either “a” or “b”. The automaton should start in an initial state and immediately transition to an accepting state upon reading the first input symbol, since any valid string must start with either “a” or “b”. After the first symbol is processed, the DFA should remain in an accepting state regardless of the subsequent sequence of “a” and “b”, thereby allowing any combination of characters to follow. Additionally, include a dead state to handle invalid or empty inputs if required by the simulator. Clearly define the states, transition rules, start state, and accepting states, and verify the design using a simulator such as Autosim to ensure that all valid strings are accepted and invalid ones are rejected.

ANSWERS

Question 1: NFA for Automatic Door System

Formal Definition (5-tuple): $N = (Q, \Sigma, \delta, q_0, F)$

- $Q = \{\text{Closed, Opening, Open, Closing}\}$
- $\Sigma = \{P, N\}$ (P = Person detected, N = No person detected)
- $q_0 = \text{Closed}$ (start state)
- $F = \{\text{Open}\}$ (accepting state — door successfully open)

Transition Table

State	P	N
$\rightarrow$ Closed	{Opening}	{Closed}
Opening	{Opening, Open}	{Open}
*Open	{Open}	{Open, Closing}
Closing	{Opening}	{Closing, Closed}

($\rightarrow$ = start state, * = final/accepting state)

Explanation of Non-determinism:

- From Opening on input P, the system may non-deterministically stay in Opening (still moving) or advance to Open — modeling the uncertain duration of the door's opening motion.
- From Open on input N, the automaton may either remain in Open (grace period / sensor delay) or transition to Closing — this is the key non-deterministic choice described in the problem.
- From Closing on input N, the system may stay Closing or complete the cycle back to Closed.

State Diagram (textual):

Closed $\xrightarrow{P}$ *Opening* $\xrightarrow{P}$ *Open* $\xrightarrow{N}$ *Closing* $\xrightarrow{N}$ *Closed*, with self-loops: *Closed* on *N*, *Opening* on *P* (self-loop, non-det.), *Open* on *P* (self-loop) and on *N* (self-loop, non-det.), *Closing* on *N* (self-loop, non-det.).

Question 2: Finite Automaton for Traffic Light Control System

Formal Definition: $M = (Q, \Sigma, \delta, q_0, F)$

- $Q = \{\text{Red, Green, Yellow}\}$
- $\Sigma = \{T\}$ (T = timer signal / timeout event that triggers the next transition)
- $q_0 = \text{Red}$ (start state)
- $F = Q$ (every state is a valid/accepting operating state — this is a cyclic controller, not a language acceptor)

Transition Table

State	Input T (timer expires)	Duration
→ Red	Green	30 sec
Green	Yellow	25 sec
Yellow	Red	5 sec

Cycle:

Red $\xrightarrow{T}$ Green $\xrightarrow{T}$ Yellow $\xrightarrow{T}$ Red $\xrightarrow{T}$... (repeats indefinitely; the automaton has no final halting state since traffic control runs continuously).

Handling continuous cycling:

Each state has an internal timer. Once the timer for a state expires, input symbol T is generated internally, forcing a deterministic transition to the next state in the fixed sequence. Because δ is a total function returning exactly one next state for T , the automaton is deterministic and cycles forever without ever reaching a dead or terminal condition.

Extension – Pedestrian Crossing:

Add a new state `Pedestrian_Walk`, reachable from Red on a pedestrian-button input signal B : $\delta(\text{Red}, B) = \text{Pedestrian_Walk}$. This state runs its own timer (walk time) and then returns to Red before the normal cycle resumes: $\delta(\text{Pedestrian_Walk}, T) = \text{Red}$.

Extension – Emergency Vehicle Override:

Add an input symbol E (emergency signal) that can interrupt any state: $\delta(\text{Green}, E) = \text{Red}$ and $\delta(\text{Yellow}, E) = \text{Red}$. This forces the intersection to Red immediately regardless of the current timer, after which a dedicated Emergency state can hold Red for all conflicting directions until E is withdrawn, then normal cycling resumes from Red.

Question 3: DFA Accepting Strings Ending with “ab” over {a, b}

Formal Definition: $M = (Q, \Sigma, \delta, q_0, F)$

- $Q = \{q_0, q_1, q_2\}$
- $\Sigma = \{a, b\}$
- q_0 = start state (no relevant suffix seen yet)
- $F = \{q_2\}$ (last two symbols read were “ab”)

Transition Table

State	a	b
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
$*q_2$	q_1	q_0

State meaning:

- q_0 — initial state / last symbol was not ‘a’
- q_1 — last symbol read was ‘a’ (potential start of “ab”)
- q_2 — accepting state; the last two symbols read were exactly “ab”

Trace for $W = \text{aaabab}$:

Step	Input	Current State	Next State
1	a	q_0	q_1
2	a	q_1	q_1
3	a	q_1	q_1
4	b	q_1	q_2
5	a	q_2	q_1
6	b	q_1	q_2

Result: Final state reached = $q_2 \in F \rightarrow$ the string “aaabab” is ACCEPTED (it ends with “ab”).

Question 4: DFA Accepting Exactly {"c", "bc", "bcaaa"} over {a, b, c}

Formal Definition: $M = (Q, \Sigma, \delta, q_0, F)$

- $Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_d\}$
- $\Sigma = \{a, b, c\}$
- q_0 = start state
- $F = \{q_1, q_3, q_6\}$ (accept exactly "c", "bc", "bcaaa" respectively)
- q_d = dead/trap state for any string that deviates from the three valid patterns

Transition Table

State	a	b	c	Meaning
$\rightarrow q_0$	q_d	q_2	q_1	start
$*q_1$	q_d	q_d	q_d	accepts "c"
q_2	q_d	q_d	q_3	read 'b'
$*q_3$	q_4	q_d	q_d	accepts "bc"
q_4	q_5	q_d	q_d	read "bca"
q_5	q_6	q_d	q_d	read "bcaa"
$*q_6$	q_d	q_d	q_d	accepts "bcaaa"
q_d	q_d	q_d	q_d	dead state (trap)

Explanation:

- $q_0 \xrightarrow{c} q_1$ accepts the exact string "c"; any symbol after reaching q_1 sends the automaton to q_d since a longer string is no longer equal to "c".
- $q_0 \xrightarrow{b} q_2 \xrightarrow{c} q_3$ accepts the exact string "bc".
- $q_3 \xrightarrow{a} q_4 \xrightarrow{a} q_5 \xrightarrow{a} q_6$ accepts the exact string "bcaaa" (exactly three a's after "bc", no more, no less).
- Every other symbol combination (wrong branch, extra symbols, or premature stop that isn't $q_1/q_3/q_6$) leads to q_d , which loops on all inputs and is never accepting — guaranteeing only the three target strings are accepted.

Question 5: DFA Accepting All Strings Starting with ‘a’ or ‘b’ over {a, b}

Formal Definition: $M = (Q, \Sigma, \delta, q_0, F)$

- $Q = \{q_0, q_1\}$
- $\Sigma = \{a, b\}$
- q_0 = start state (nothing read yet)
- $F = \{q_1\}$ (at least one symbol has been read)

Transition Table

State	a	b
$\rightarrow q_0$	q_1	q_1
$*q_1$	q_1	q_1

Explanation:

Since $\Sigma = \{a, b\}$, every non-empty string necessarily begins with either ‘a’ or ‘b’. As soon as the first symbol is read, the DFA moves from q_0 to the accepting state q_1 and remains there (self-loop on both a and b) for the rest of the string, regardless of what follows. The empty string leaves the automaton in q_0 , which is non-accepting, so it is correctly rejected.

Note on the dead state: A separate trap state is not strictly required here because both symbols from both states lead to a valid accepting state; if the simulator (e.g., Autosim) requires δ to be fully defined and a designated fail case for malformed/non-alphabet input, an additional q_d state can be added such that $\delta(q_0, x) = q_d$ and $\delta(q_1, x) = q_d$ for any $x \notin \{a, b\}$.